

1.1 - Overview and Design

This report will explain an algorithm designed to detect and classify bottle caps placed on a surface. The surface has a reference frame visible in which each bottle cap's position and radius will be outputted in relation to. The system will also detect the colour of each bottle cap and report that as well. It will output a .dat file containing all of this information as well as optionally display figures of all the intermediate steps with the `SHOW_FIGURES` variable at the beginning of the function script. The outlined solution uses traditional image processing techniques including segmentation, feature extraction and classification. It does not use any machine learning algorithms. An overview of the algorithm steps can be seen in Figure 1.

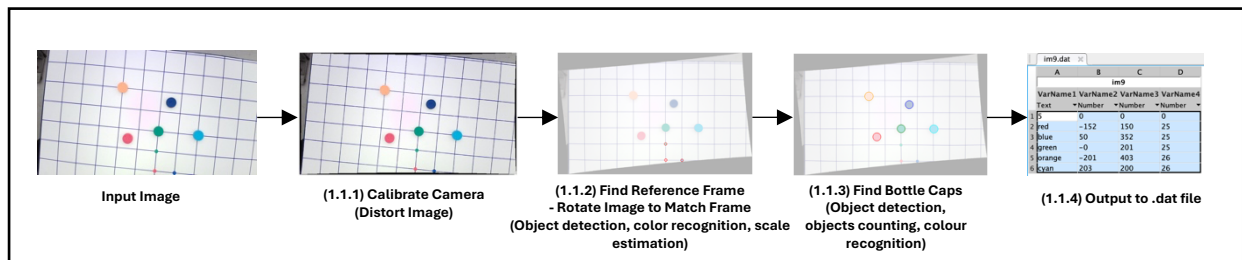


Figure 1. System Overview Block Diagram

1.1.1 - Camera Calibration

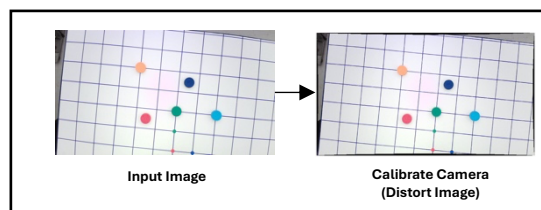


Figure 2. Camera Calibration Block Diagram

Looking at the original images supplied in the challenge, it can be seen that there is a slight radial distortion that occurs as you go towards the edges of the images. This causes the grid pattern which we know to be straight to appear distorted. Without calibrating the camera, finding the (x, y) positioning of the bottle caps will yield results that are not accurate because of the warping that occurs. Every camera has its own intrinsic properties which explains how this distortion occurs. We were not provided this information for this project so I estimated the information to the best

of my abilities to correct the images. Looking at the image I assumed there was only Barrel distortion. K represents the intrinsic matrix. Looking at the camera specs online I found it to have a focal length of 4.4mm and a sensor size of 9.407mm.

$$f_{x,y} (FocalLength - px) = actualFocalLength * imgSize_{x,y} / sensorSize_{x,y}$$

$$c (PrinciplePoint) = [imgSize_x/2 \quad imgSize_y/2]$$

$$radialDistortion = [-0.5 \quad 0.2]$$

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 4224 & 0 & 960 \\ 0 & 4752 & 540 \\ 0 & 0 & 1 \end{bmatrix}$$

It is important to note that the radial distortion was chosen through trial and error to attempt to straighten the grid in the image and resulted in a decent approximation but is not exact. Once we have this intrinsic matrix and the radial distortion calculated we can use the `undistortImage()` function, plugging in these values to warp the image and correct for the barrel distortion. This will allow the grid lines to be straight and thus allow for a more accurate pixel to real-world coordinate transformation to find the actual position of the bottle caps in relation to the reference frame. Since the instructions said that we can assume that that camera will always be pointing straight down, we can ignore any effects of perspective projection because the image sensor can be assumed to be parallel to the grid surface. The last step in this section is to convert the image from RGB colour space to HSV colour space. This will allow for better segmentation of colours to detect specific colours in the scene because HSV separates colour information (Hue) from intensity, making it ideal to separate colours even when the lighting/colour intensity changes.

1.1.2 – Finding The Reference Frame

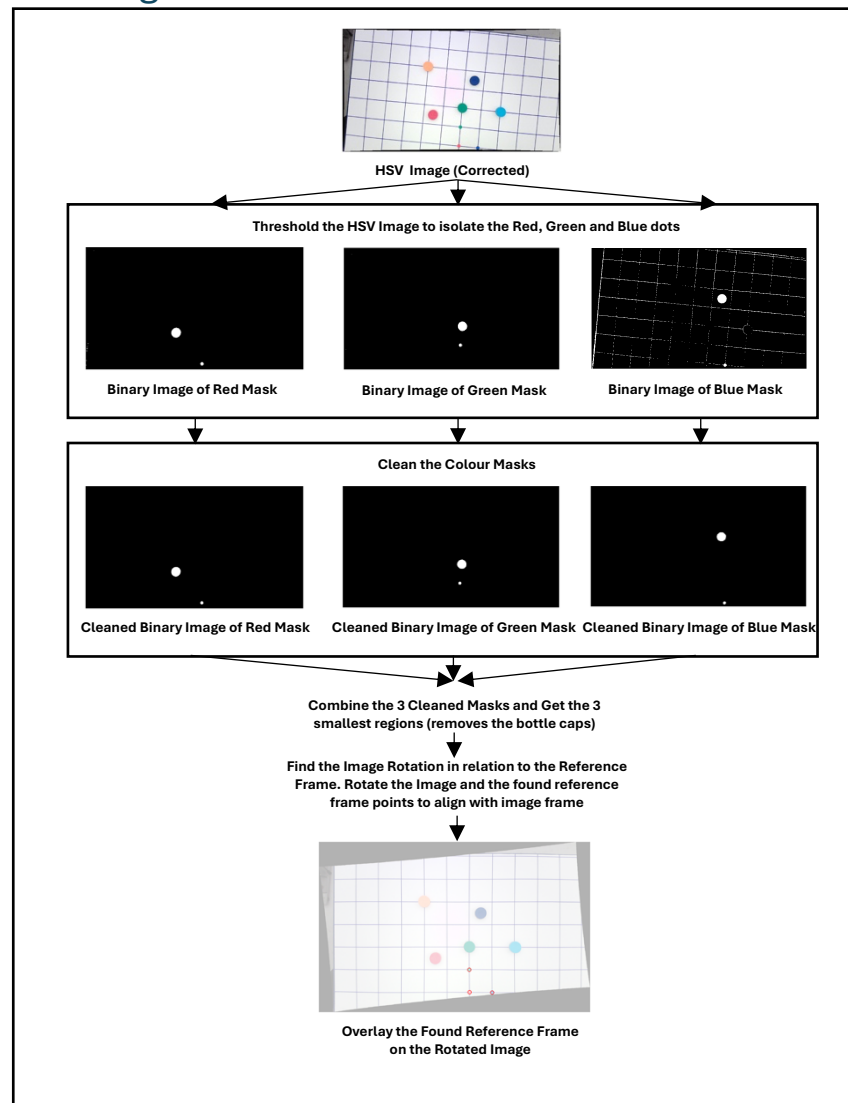


Figure 3. Finding Reference Frame Block Diagram

The next step in the algorithm is to find the reference frame dots on the HSV image that is corrected for distortion in the earlier step. These dots are defined as Red (0, 0), Green (0, 100mm) and Blue (100mm, 0) making a +Y and +X axis. The goal is to find the coordinates of the bottle caps in relation to this reference frame so we must locate these dots in the image. The way this was done is first isolating the red, green and blue channels and create a binary mask for each of these colours. To do this we define a range of Hue, Saturation and Value's to keep and create a binary mask with. The following HSV ranges are used to segment the image to find the reference frame:

- **Red:** hue range [0, 0.1], saturation > 0.5, value > 0
- **Green:** hue range [0.2, 0.5], saturation > 0.4, value > 0
- **Blue:** hue range [0.55, 0.7], saturation > 0.4, value > 0

These ranges were chosen experimentally using MATLAB's built-in imageSegmenter application which allows for selection of certain parts of an image and outputs the HSV ranges needed to segment that part of the image. Further refinement through experimentation was performed after this step to achieve a better binary mask given lighting changes between images.

Once we have the 3 binary masks, we can then perform cleaning of these masks in order to remove unwanted noise that will affect our ability to detect the reference frame.

Looking at the 3 masks, it is clear that there is much more unwanted noise in the blue binary mask compared to the red and green masks. This is due to the fact that the grid in the original image is blue which gets picked up by the blue binary mask. Because of this, the algorithm was developed to locate the red and green reference dots first, then use the size of those dots to filter the blue mask to remove noise less than the radius of the already found red and green reference frame dots. To find the size of the red and green reference frame dots we first label all connected regions with the `bwlabel()` function. This returns a list of labeled regions (connected pixels) and a count of the number of regions in the image. We then go through all the regions, calculating the radius of each of these regions using the `regionprops()` function which calculates the Major and Minor axis length of the region. The radius can then be calculated by taking the mean of these two values and dividing by 2. Once we have all the radii of the regions in the image we can sort them and take the smallest radii which will be our reference frame. This is done for the red and green binary masks so that the blue mask has access to these radii to remove the grid pattern. This step will remove all of the blue grid pattern from the blue segmented binary image and produces a result containing only the blue reference frame and any blue bottle caps. This dynamic adjustment of the size of noise removal allows the solution to work even when the camera is further from the table (i.e reference frame dots have a lower resolution). The noise removal step uses MATLAB's morphological operations with the `imopen()` and `strel(disk)` functions. The `imopen()` function uses erosion followed by dilation to remove pixels using a structuring element defined by the `strel(disk)` function. Since we know that the reference frame is circles we can use a disk with radius of the reference frame that was found in the red and green binary masks. The implemented solution subtracts 3px from this average radius to account for variations in the sizes of the reference frame dots in px. This will remove pixel groups less than with $\text{radius} < (\text{redRadius} + \text{greenRadius})/2 - 3\text{px}$ from the blue binary mask. To clean the masks further we apply another `bwlabel()` on the cleaned masks, labelling each region again and finding the Circularity of each region `regionprops()` function. This function outputs a value that represents the circularity of every region. The circularity can be from 0 to 1, with 1 being the max circularity. We go through each region and remove all regions that are not circular enough (less than a threshold of 0.7 chosen experimentally). This removes unwanted lines or noise that are not the reference frame dots.

To find the reference frame in these cleaned binary masks we combine the 3 cleaned binary images and take the 3 smallest area regions which represent the reference frame dots. To do this we use the same technique mentioned earlier of labeling the connected regions in the image, finding each regions radius and sorting based on the radius. We now have the centroid pixel locations of the 3 reference frame dots.

The next step is to find the orientation of this reference frame in relation to the image frame in order to correct for any rotation. This ensures that the reference frame coordinate system matches the image coordinate system for when we try to locate the location of the bottle caps later. To do this we find the vectors pointing from the red to the green (V_y) and from the red to the blue (V_x) centroids. We simply just subtract the coordinates of the red reference centroid from both of these. This gives us 2 vectors which represent the orientation of the x and y axis in relation to the image frame. To find the angle of both of these vectors in relation to the image frame we use this formula:

$$\theta_{V_x, V_y} = \tan^{-1} \left(\frac{v_{x,y}(y)}{v_{x,y}(x)} \right)$$

Where θ_{V_x, V_y} represents the angle of either the +X vector or +Y vector.

$v_{x,y}(x)$ and $v_{x,y}(y)$ represents the x and y components of the respective vector. Once both of these angles are found we can add 90° to the angle of the Y axis vector since we know they will be ~90° from each other. Now the two angles should be very close to each other but won't be exactly the same due to lens distortions that were not corrected fully in the first step. We average the two angles to get an approximation for the rotation needed to get this reference frame lined up with the image frame. There is also a quick check here if the image is upside down by checking if the y component of the V_y vector is upside down. In this case we add 180° to the angle to correct for this upside down image. Now that we have this angle we can rotate the image and the located centroids by this angle. Rotating the image is done with the `imrotate()` function where the angle is passed into the function. Rotating the reference centroids that we found earlier requires the use of a rotation matrix. The rotation matrix is defined as follows, where "angle" represents the rotation angle of the reference frame we have just found:

$$R = \begin{bmatrix} \cos(\text{angle}) & -\sin(\text{angle}) \\ \sin(\text{angle}) & \cos(\text{angle}) \end{bmatrix}$$

We then apply this transformation matrix to the found reference frame centroids (x, y) location to rotate them as well. We then recalculate V_x and V_y for use later use. They should now be pointing close to directly horizontal and close to vertical. Lastly, we overlay and display an image of red circles on top of the original image with this new rotation to ensure that the detected reference frame lines up with the reference frame circles visually in the image.

1.1.3 – Finding Bottle Caps

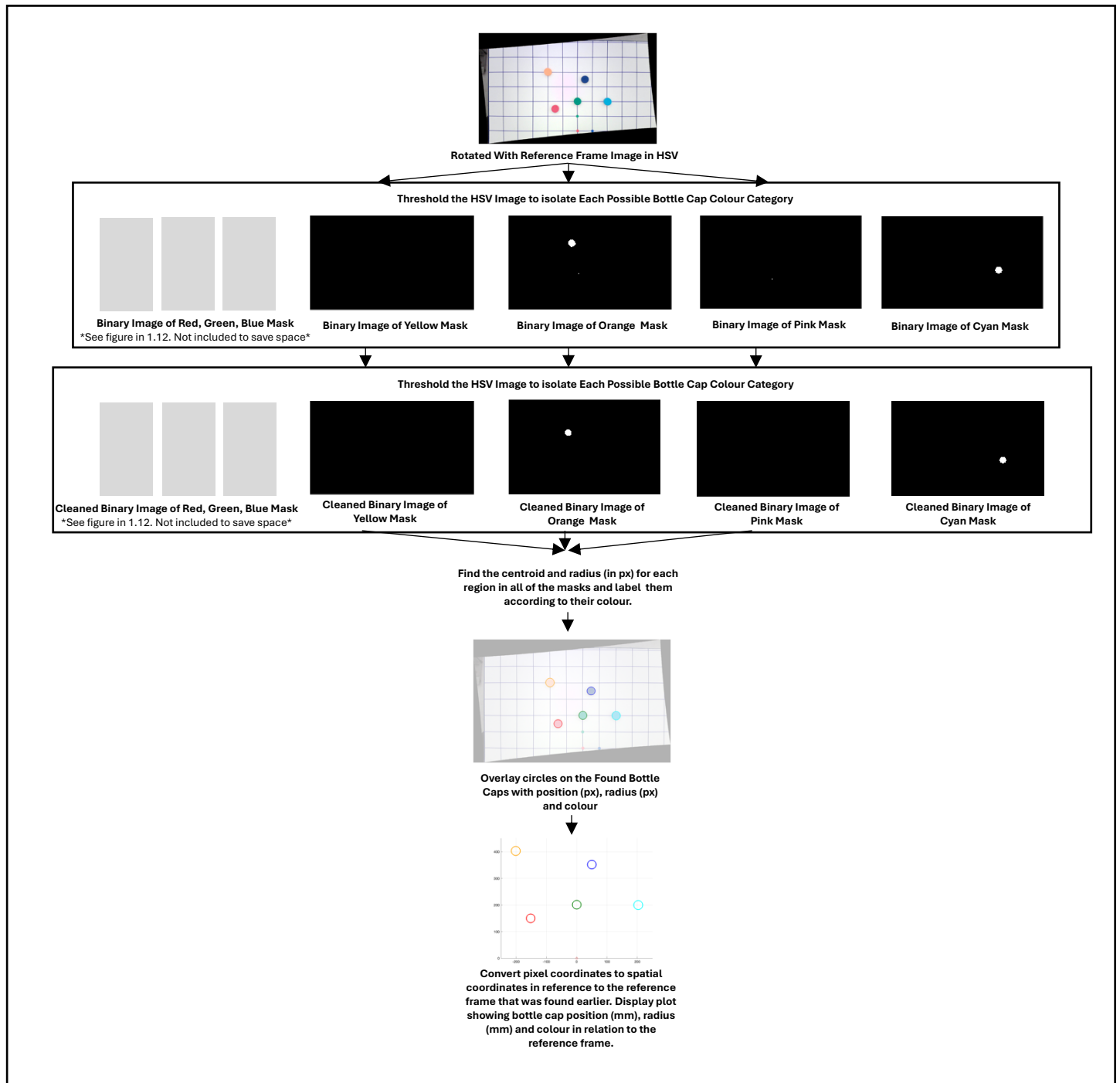


Figure 4. Finding Bottle Caps Block Diagram

Once we have located the reference frame and rotated the image so the reference frame is aligned with the image frame, we can start to locate the bottle caps in the

image. The algorithm first segments the corrected image based on the defined possible colours of the bottle caps. It does this in a similar way to how it was done in Section 1.1.2 except we define a range for the saturation and value channels in addition to the hue channel. This gives more accuracy in the colours we segment that wasn't needed for the simple red, green and blue reference frame. The following HSV ranges were used to segment each of the pre-defined colours and were chosen experimentally using the imageSegmenter application in MATLAB as well:

- **Red:** hue range [0.9, 1.0], saturation range [0.5, 1.0], value range [0.4, 1.0]
- **Blue:** hue range [0.55, 0.7], saturation range [0.5, 1.0], value range [0.4, 1.0]
- **Green:** hue range [0.2, 0.5], saturation range [0.5, 1.0], value range [0.4, 1.0]
- **Yellow:** hue range [0.1, 0.3], saturation range [0.2, 1.0], value range [0.4, 1.0]
- **Orange:** hue range [0.98, 0.1], saturation range [0.2, 0.5], value range [0.8, 1.0]
- **Pink:** hue range [0.88, 0.95], saturation range [0.16, 0.35], value range [0.8, 0.99]
- **Cyan:** hue range [0.51, 0.57], saturation range [0.6, 1.0], value range [0.5, 1.0]

Similarly to how we removed noise from the noisy blue channel in Section 1.1.2, we now have stored the radius of the reference frame dots from earlier. Since we want to remove all regions that are smaller than the size of a bottle cap, we can remove all regions that have a radius less than or equal to the radius of the found reference frame circles (+3px buffer). We do this by first filling any holes in the binary regions with `imfill()` (this makes the extracted regions more continuous). Then we do the same operation that was done in section 1.1.2 with the `imopen()` function and a disk structured element with $\text{radius} = (\text{redRadius} + \text{greenRadius})/2 + 3\text{px}$. This insures that all regions less than the radius of the reference frame dots, +3px buffer, are removed. This leaves a clean image for each of the detected colour masks that only includes the bottle caps.

The algorithm then goes through each of the binary masks for each channel and labels each region, storing the (x,y) coordinates, radius and colour (based on the channel name). It does this the same way that was described in 1.1.2, using `bwlabel()` and `regionprops()`.

Once we have these (x,y) pixel coordinates of each centroid, we can find the coordinates of each bottle cap in relation to the detected reference frame from earlier by first scaling the pixel coordinates to the known grid spacing in mm (from the reference frame). To find the scaling factor in the x and y directions we use the following equation which represents the scaling in the x or y directions:

$$Scale_{x,y} = \frac{GridSpacing_{x,y}}{norm(V_{x,y})}$$

$$GridSpacing_{x,y} = 100 \text{ mm} \rightarrow \text{From instructions}$$

$$norm(V_{x,y})$$

= *Calculates the pixel length of the vector on the x or y reference frame*

The scaling factor in the x and y directions should be very close to each other but will not be perfectly equal due to lens distortions that were not fully corrected in Section 1.1.1.

We can now simply subtract the red reference centroid pixel location (0, 0) from each detected bottle cap pixel location. This gives us the (x,y) pixel locations of the bottle caps in relation to the red origin point on the reference frame. To convert these pixel values into mm values, we multiply each (x, y) pixel value by the scaling factor calculated earlier $Scale_x$ or $Scale_y$. We do the same with the radii which was stored in units of pixels earlier. We multiply each radii value by the average scaling factor in the x and y directions $(Scale_x + Scale_y)/2$. We now have the coordinates of each bottle cap in relation to the reference frame stored in mm. The algorithm overlays each found bottle cap centroid location, radii and colour on the original rotated image in order to confirm visually all bottle cap locations, radii and colours were found accurately. It also outputs a plot showing the bottle caps positioning, radius and colour in relation to the red reference origin point (0,0).

1.1.4 – Outputting to .dat file

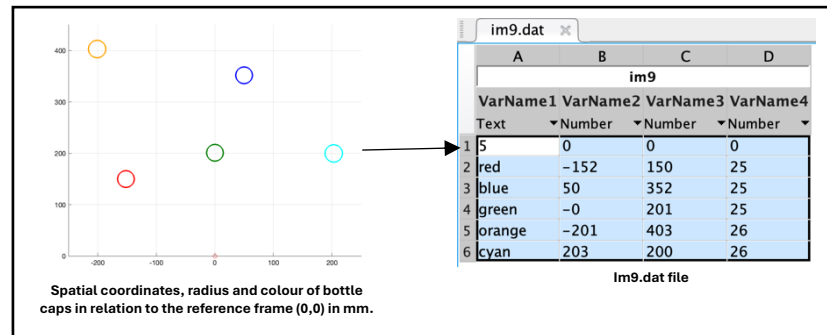


Figure 5. Finding Bottle Caps Block Diagram

Since we already have stored the position (in mm in reference to the red reference origin point), radius (mm) and colour of each of the detected bottle caps, the algorithm simply loops through all of these detected caps to store each bottle cap on its own line in a .dat file. The first line contains the number of bottle caps detected, followed by three 0's. The next lines contain information about the colour, x (mm), y (mm) and radius (mm) of each bottle cap in relation to the red reference origin at (0,0).

1.2 - Discussion

Visually comparing each of the supplied images with the output from each of the .dat files after running the script, it can be seen that the solution outlined in this report successfully locates and classifies all bottle caps in all test images supplied. In all test images, the bottle count outputted by the script matches the count when observing the image visually. The detected colours and radius of each of the bottle caps can also be seen by looking at each figure in Appendix A, where a circle outlining the bottle cap matching the detected colour is overlaid on the original image. The HSV ranges used for detecting the colour classes of the bottle caps were chosen to work with these images. It was found that the HSV values of the Red, Orange and Pink bottle caps are fairly close to each other and were more error prone when trying to get an accurate binary mask of one specific colour class. This risk was reduced by implementing a range for the saturation and value channels in addition to the hue channel in order to dial into the specific colour range desired. This was explained in Section 1.1.3.

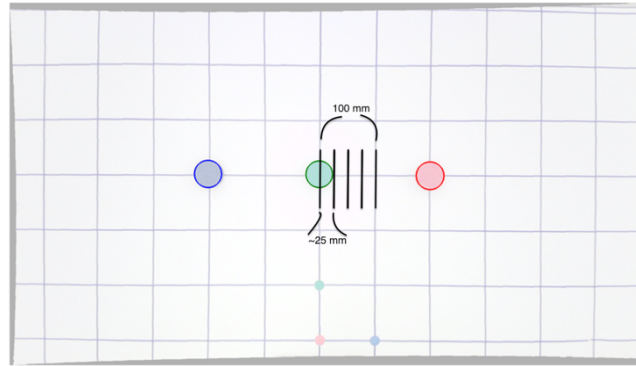


Figure 6. Expected Bottle Cap Radius

From Figure 6 it can be seen that the expected radius of the bottle caps is ~25mm given that the supplied grid has a spacing of 100mm. Splitting this distance into 4 equal segments in the image shows that the radius of the bottle cap matches a $100\text{mm}/4$ segments = 25mm / segment. This is a rough estimate but gives an idea of the range of expected radii we should get in our output file. Looking at the results for each image seen in Appendix A, it can be seen that each detected radius is fairly close to 25mm. The outputted x and y coordinates of each bottle cap in relation to the red reference frame at (0,0) are all very close to the expected values when comparing the outputted results with a visual inspection of the images. This visual inspection confirmation was done using the grid lines to estimate the positioning of the bottle caps in the scene and comparing this to the outputted results from the script.

In addition to the supplied images, further testing was done with modified images to ensure that the script works in more scenarios. Examples of this can be found in Appendix B. These images include a reduced resolution image and an image with the reference frame upside down. The reduced resolution image is meant to simulate if the camera was placed up higher (further from the table) so the bottle caps and reference frame dots would appear to have smaller radii (in px). The reduced resolution image is the same as im4.jpg but has a resolution of 700x394 px. The algorithm successfully finds all bottle caps position, radius and colour with this reduced resolution image and the positioning and radius is very close to the values found after running the script on the original image. The next photo with the image upside-down is the same as im9.jpg but rotated 180°. This is meant to simulate if the camera +Y does not match the reference frame +Y (upside-down). The algorithm successfully finds all bottle caps position, radius and colour with this test image and gives very similar results to that of the original im9.jpg image.

Although the solution outlined in this report performs well with all the supplied images, as well as some additional testing images, it may not perform well if the scene becomes more complicated. An example of this is if the grid pattern doesn't take up the whole image and

there is random objects visible around the perimeter of the image. In the thresholding step, the algorithm could possibly go wrong if it finds objects off of the grid that match the threshold colour mask criteria. In this case the solution would probably fail. To fix this issue, further development of the solution could be made to detect the boundary of the grid region and ignore objects that are outside of this area. Another source of error in the positioning calculations is the fact that the script makes a lot of assumptions about the camera parameters (as described in Section 1.1.1). This causes small discrepancies in the bottle cap radii and positioning. This could be reduced by having a more accurate camera calibration at the beginning of the script. This could be done using MATLAB's camera calibration feature where it automatically calculates the camera parameters to undistort the image using an test image with a checkerboard pattern in the scene. Another improvement could be made in the colour of the grid pattern. As seen in Section 1.1.2, the blue grid pattern makes it difficult to locate the blue reference frame dot in the scene since. Further iterations of this project could use a black grid pattern in order to not interfere with the blue circle detection and provide a more accurate result.

1.3 - Conclusion

Overall the solution outlined in this report successfully implements a MATLAB function that can locate and classify bottle caps in the supplied sample images using traditional image processing techniques. In addition to this, the system was made to robustly handle different situations including the lowering of the image resolution and rotation of the reference frame in relation to the camera frame. Although the solution works well and gives accurate results for the images provided, it may perform poorly on additional images where there are more complicated scene features to take into account. Further refinement and improvement of the solution can be made including proper camera calibration and detection of the grid to ignore objects outside of the grid region.

Appendix A – Results for Sample Test Images

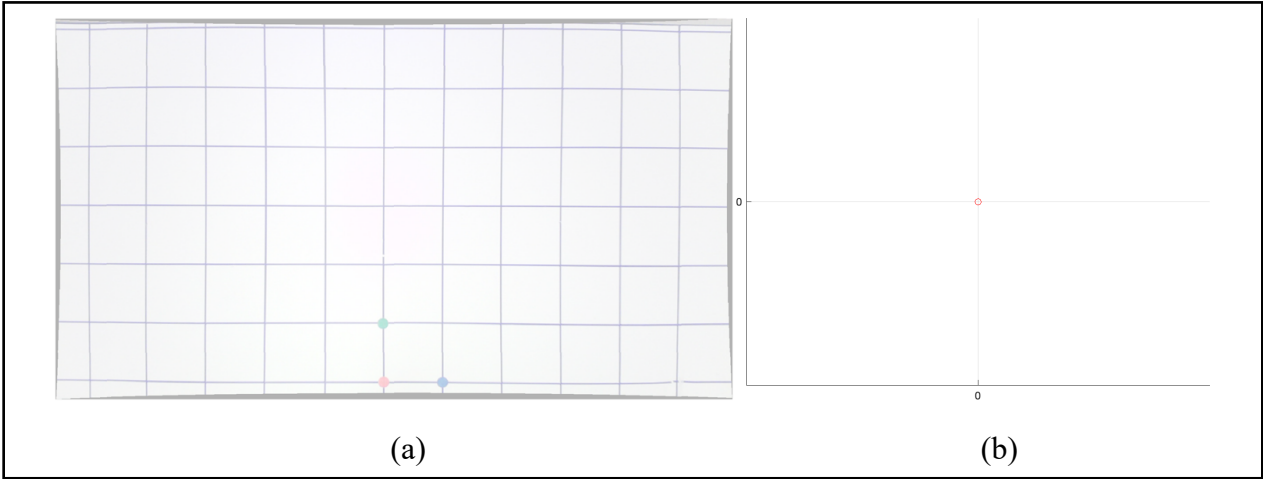


Figure 7. Located bottle caps for im1.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im1.dat				
A				
B				
C				
D				
im1				
VarName1 VarName2 VarName3 VarName4				
Number Number Number Number				
1	0	0	0	0

Figure 8. im1.dat

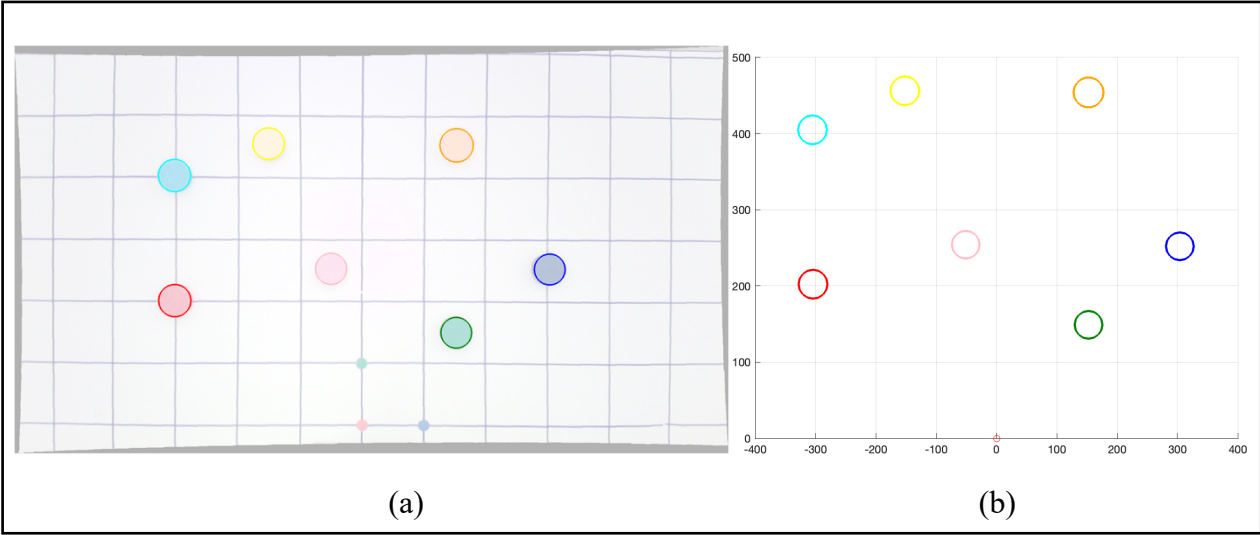


Figure 9. Located bottle caps for im2.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im2.dat				
A B C D				
im2				
VarName1	VarName2	VarName3	VarName4	
Text	Number	Number	Number	
1 7	0	0	0	
2 red	-305	202	26	
3 blue	305	253	25	
4 green	153	149	25	
5 yellow	-152	457	26	
6 orange	153	455	27	
7 pink	-51	254	25	
8 cyan	-306	406	26	

Figure 10. im2.dat

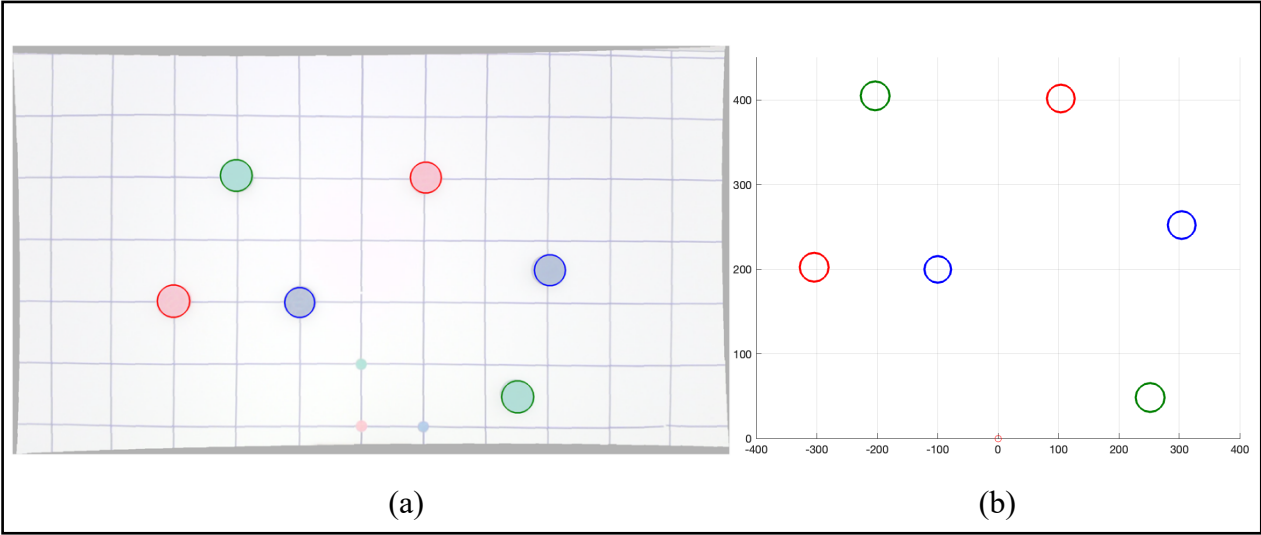


Figure 11. Located bottle caps for im3.jpg (a) Cap position, colour and radius overlayed on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im3.dat			
A	B	C	D
im3			
VarName1	VarName2	VarName3	VarName4
Categori...	Number	Number	Number
1	6	0	0
2	red	-305	202
3	red	104	402
4	blue	-100	200
5	blue	305	252
6	green	-203	406
7	green	252	47

Figure 12. im3.dat

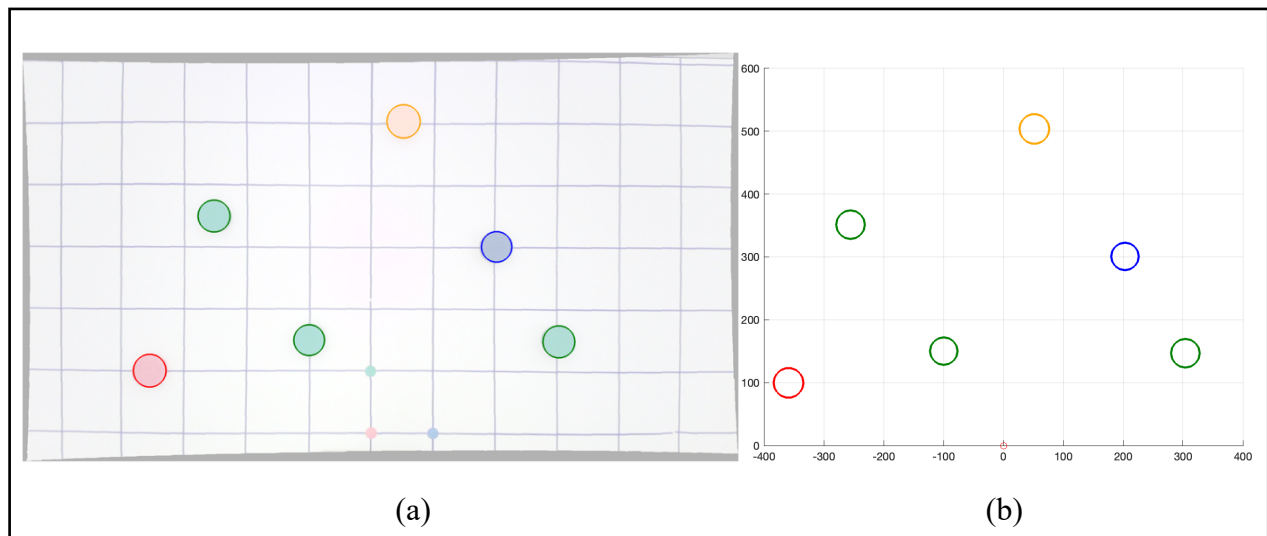


Figure 13. Located bottle caps for im4.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im4.dat				
	A	B	C	D
	im4			
	VarName1	VarName2	VarName3	VarName4
	Text	Number	Number	Number
1	6	0	0	0
2	red	-361	100	27
3	blue	204	301	25
4	green	-255	351	26
5	green	-101	150	25
6	green	305	147	26
7	orange	53	504	27

Table 14. im4.dat

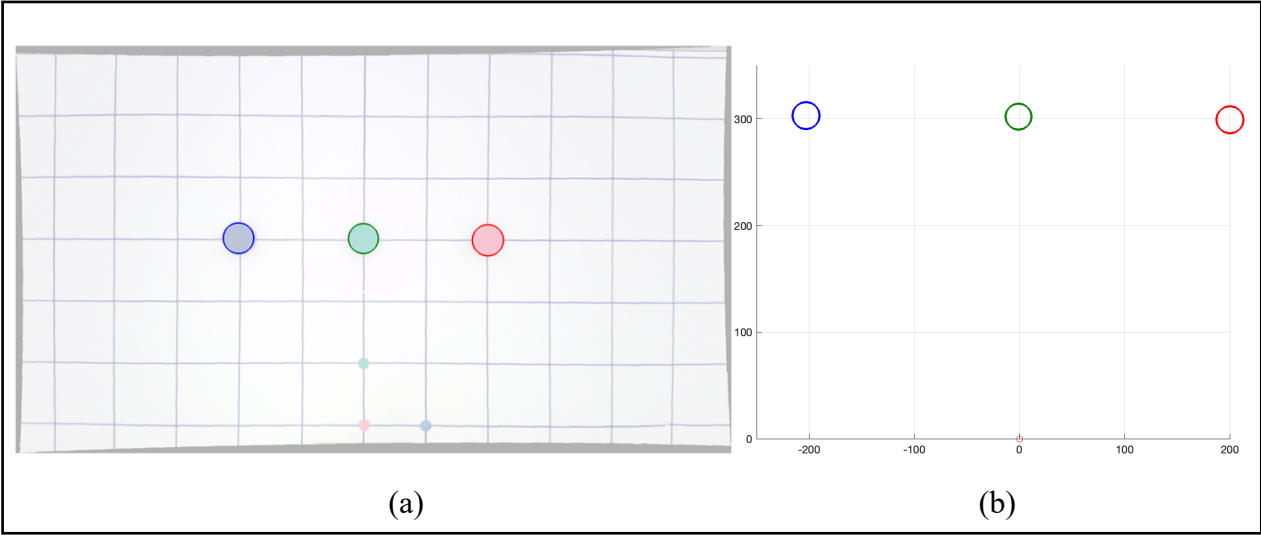


Figure 15. Located bottle caps for im5.jpg (a) Cap position, colour and radius overlayed on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im5.dat				
A B C D				
im5				
VarName1 VarName2 VarName3 VarName4				
Text Number Number Number				
1	3	0	0	0
2	red	200	300	25
3	blue	-204	303	25
4	green	-2	303	24

Figure 16. im5.dat

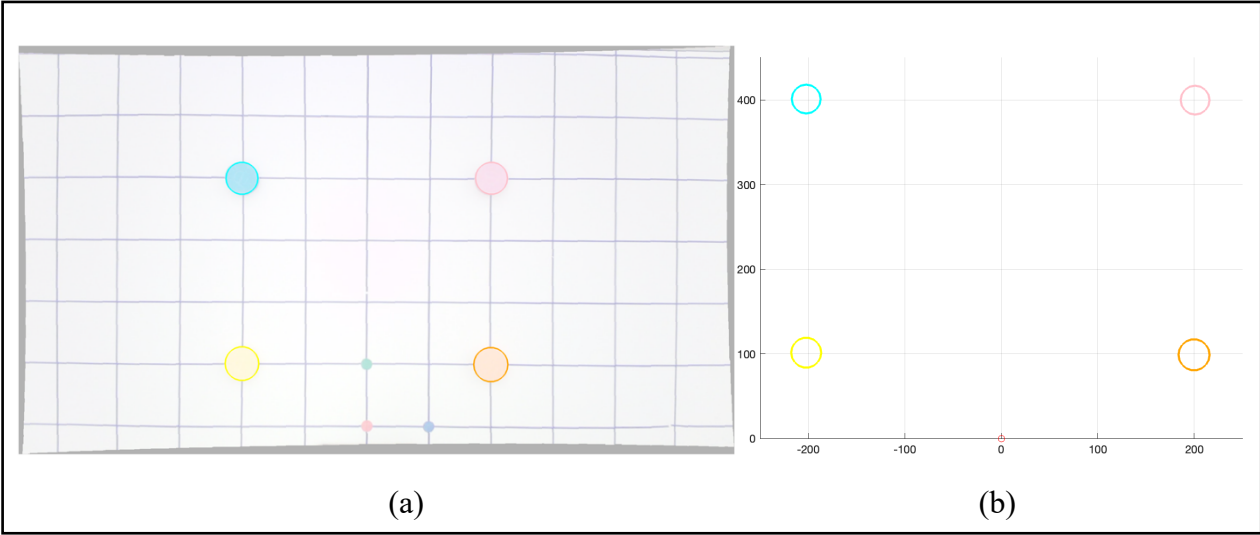


Figure 17. Located bottle caps for im6.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im6.dat				
	A	B	C	D
im6				
	VarName1	VarName2	VarName3	VarName4
	Text	Number	Number	Number
1	4	0	0	0
2	yellow	-203	101	27
3	orange	201	99	28
4	pink	201	402	26
5	cyan	-204	401	26

Figure 18. im6.dat

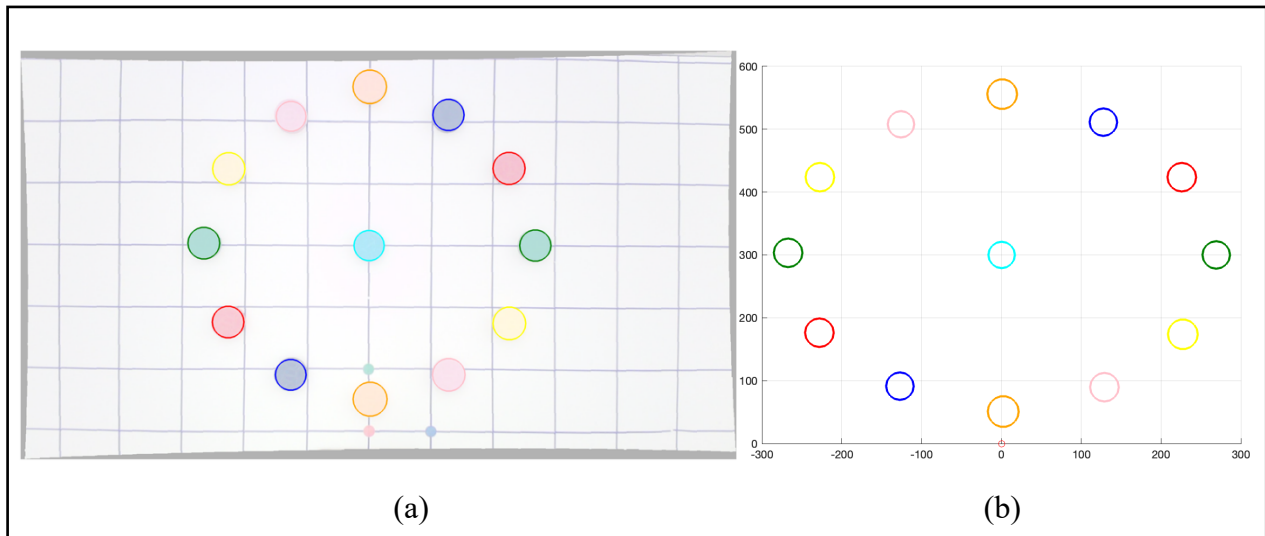


Figure 19. Located bottle caps for im7.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im7.dat				
	A	B	C	D
	im7			
	VarName1	VarName2	VarName3	VarName4
	Categori...	Number	Number	Number
1	13	0	0	0
2	red	-229	176	26
3	red	227	425	26
4	blue	-127	90	25
5	blue	128	512	25
6	green	-268	304	26
7	green	269	300	25
8	yellow	-228	425	26
9	yellow	227	174	27
10	orange	1	557	27
11	orange	2	51	27
12	pink	-127	509	24
13	pink	129	90	26
14	cyan	-0	300	24

Figure 20. im7.dat

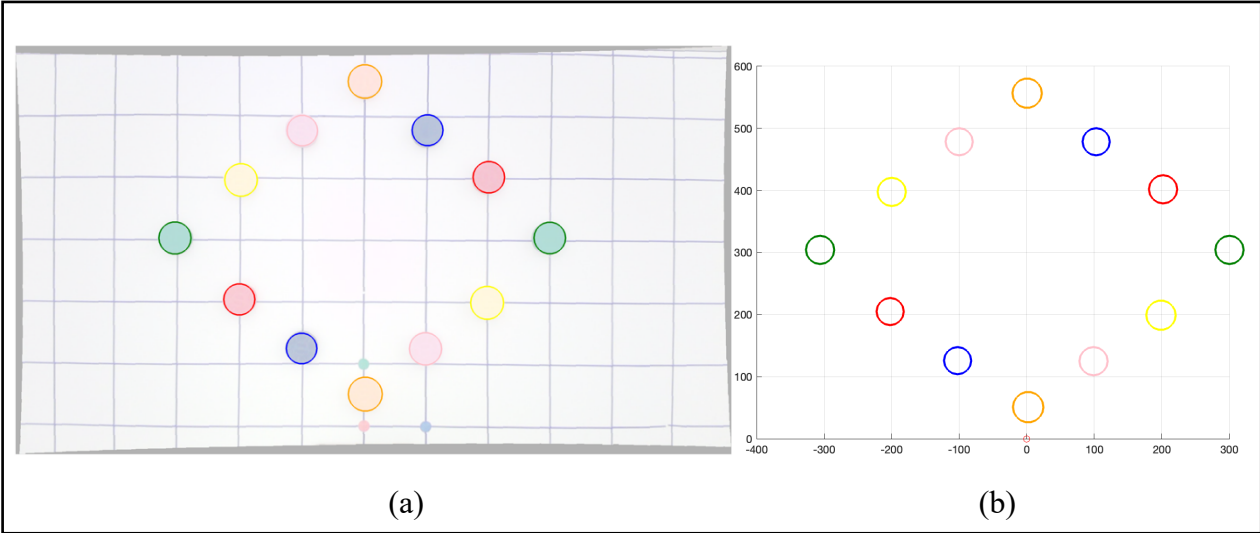


Figure 21. Located bottle caps for im8.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im8.dat				
	A	B	C	D
im8				
	VarName1	VarName2	VarName3	VarName4
	Categori...	Number	Number	Number
1	12	0	0	0
2	red	-203	205	25
3	red	202	403	26
4	blue	-102	125	25
5	blue	103	479	25
6	green	-308	304	26
7	green	301	304	26
8	yellow	-200	399	27
9	yellow	200	199	27
10	orange	2	52	28
11	orange	1	557	27
12	pink	-101	478	25
13	pink	100	125	26

Figure 22. im8.dat

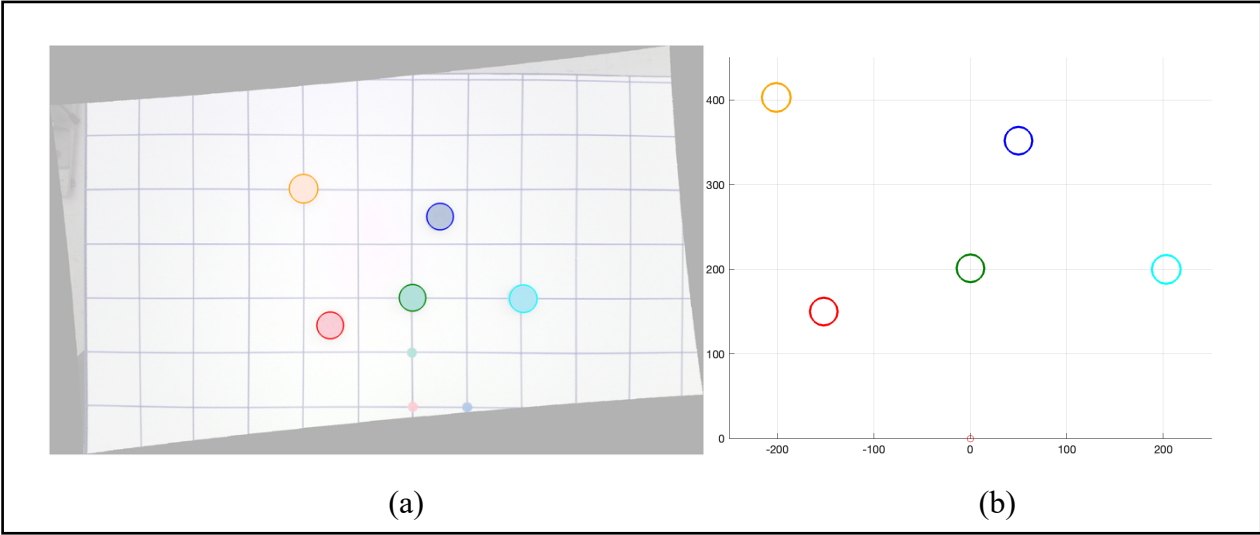


Figure 23. Located bottle caps for im9.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im9.dat				
	A	B	C	D
	im9			
	VarName1	VarName2	VarName3	VarName4
	Text	▼Number	▼Number	▼Number
1	5	0	0	0
2	red	-152	150	25
3	blue	50	352	25
4	green	-0	202	25
5	orange	-201	404	27
6	cyan	203	200	26

Figure 24. im9.dat

Appendix B – Results for Additional Test Images

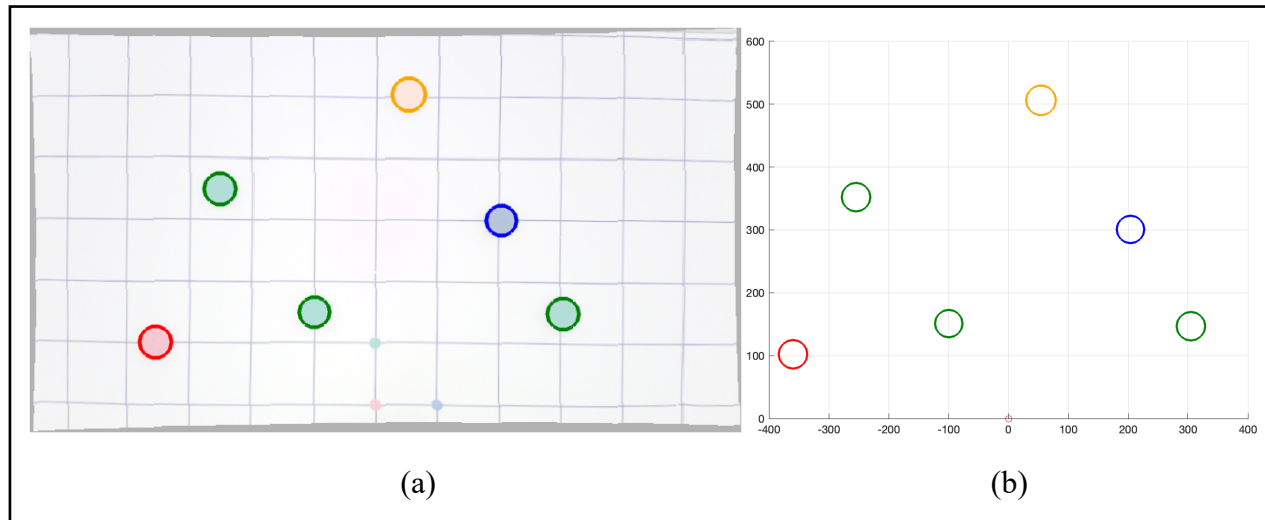


Figure 25. Located bottle caps for im4_lowRes.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im4_lowRes.dat				
	A	B	C	D
im4lowRes				
VarName1	VarName2	VarName3	VarName4	
Text	Number	Number	Number	
1 6	0	0	0	
2 red	-360	100	26	
3 blue	203	301	25	
4 green	-256	350	26	
5 green	-101	150	25	
6 green	305	148	26	
7 orange	52	504	27	

Figure 26. im4_lowRes.dat

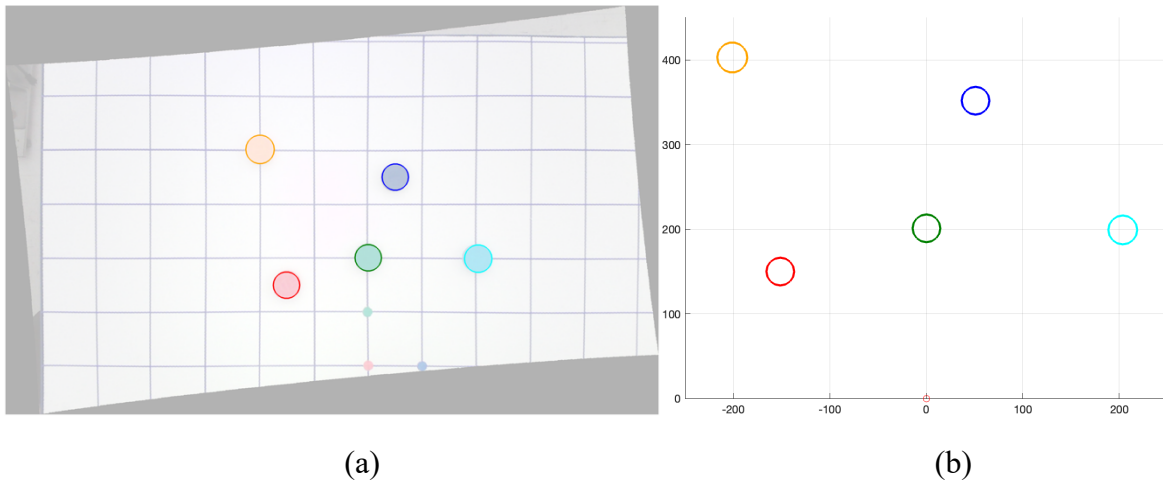


Figure 27. Located bottle caps for im9_rotated180.jpg (a) Cap position, colour and radius overlaid on original corrected image, (b) Cap position, colour and radius in relation to reference frame at (0, 0)

im9_rotated180.dat				
	A	B	C	D
	im9rotated180			
	VarName1	VarName2	VarName3	VarName4
	Text	▼Number	▼Number	▼Number
1	5	0	0	0
2	red	-151	150	25
3	blue	51	352	25
4	green	0	201	25
5	orange	-201	403	27
6	cyan	204	199	26

Figure 28. im9_rotated180.dat